

CSE327

Software Engineering

AKM IQTIDAR NEWAZ [IQN]

LECTURE - 9

Strategy Design Pattern (Behavioral)

- ❑ Defines a family of algorithms and encapsulates each one and makes them interchangeable. The strategy lets the algorithm vary independently from clients that use it.
- ❑ **Use Case:**
 - ❖ Different variations of an algorithm are needed
 - ❖ The client does not need to know the algorithm
 - ❖ The algorithm should be exchangeable at the run time of a program

Key Components

❑ Context:

- ❖ **Role:** Acts as the primary class that interacts with the clients and uses a Strategy object to perform specific behaviors.
- ❖ **Responsibility:** Maintains a reference to a Strategy object and delegates the algorithm's execution.
- ❖ **Flexibility:** Can switch between different Strategy objects at runtime, enabling dynamic behavior changes.

❑ Strategy Interface:

- ❖ **Role:** Serves as a common contract for all concrete strategy implementations.
- ❖ **Responsibility:** Declares a method (or methods) that all concrete strategies must implement.
- ❖ **Purpose:** Ensures consistency and allows the Context to work with any Strategy implementation interchangeably.

❑ Concrete Strategies:

- ❖ **Role:** Provide specific implementations of the algorithm defined in the Strategy interface.
- ❖ **Responsibility:** Implement different variation of the algorithm, encapsulating their details.
- ❖ **Purpose:** Allow easy addition of new behaviors without modifying the Context or other strategies.

Example 1 (Basic)

- **Strategy**

- ❖ Defines the interface for the family of algorithms.

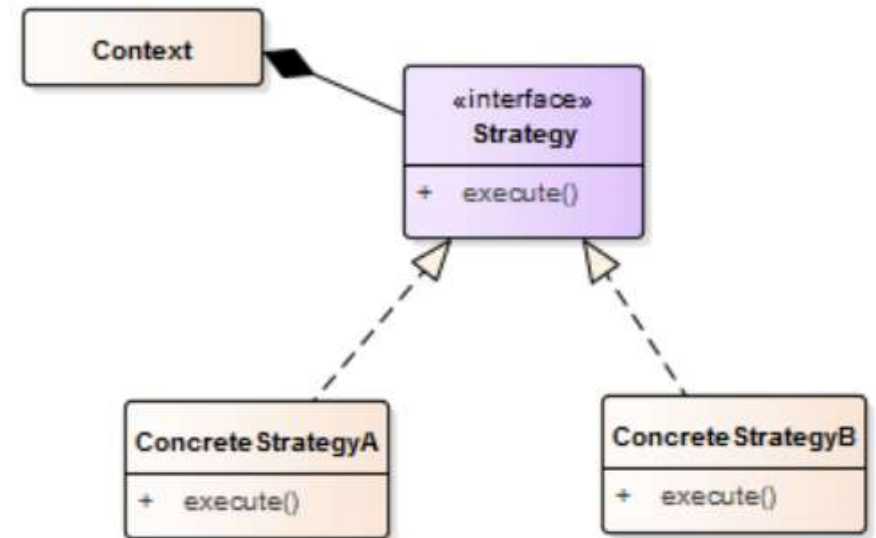
- **ConcreteStrategy**

- Implements an algorithm.

- **Context**

- ❖ Manages a reference to a ConcreteStrategy
- ❖ Has a ConcreteStrategy

- The context has a **ConcreteStrategy** that is encapsulated in an object.
- **ConcreteStrategy** implements the interface of Strategy.
- Typically, the **ConcreteStrategy** can be adjusted during run time.



Example 1 (Basic [Code])

```
#include <iostream>
#include <memory>
#include <utility>

// Interface or Abstract Class
class Strategy
{
public:
    virtual void execute() = 0;
    virtual ~Strategy() = default;
};
```

main.cpp

```
// Context holds a Concrete strategy that is encapsulated in an object
class Context
{
public:
    void setStrategy(Strategy *s)
    {
        strat = s;
    }

    void strategy()
    {
        if (strat)
        {
            strat->execute();
        }
    }

private:
    Strategy *strat;
};
```

main.cpp

Example 1 (Basic [Code])

```
// Concrete Strategy1
class Strategy1 : public Strategy
{
public:
    void execute() { std::cout << "Strategy1 executed\n"; }
};
// Concrete Strategy2
class Strategy2 : public Strategy
{
public:
    void execute() { std::cout << "Strategy2 executed\n"; }
};
// Concrete Strategy3
class Strategy3 : public Strategy
{
public:
    void execute() { std::cout << "Strategy3 executed\n"; }
};
```

main.cpp

```
int main()
{
    Context k;

    k.setStrategy(new Strategy1());
    k.strategy();

    k.setStrategy(new Strategy2());
    k.strategy();

    k.setStrategy(new Strategy3());
    k.strategy();
}
```

main.cpp

Output:

Strategy1 executed
Strategy2 executed
Strategy3 executed

Example 2 (SortSearch [Code])

```
#include <iostream>
using namespace std;

// Interface or Abstract Class
class SortBehavior
{
public:
    virtual void sort() const = 0;
};
```

main.cpp

```
class Merge : public SortBehavior
{
public:
    void sort() const override
    {
        cout << "Merge Sort()" << endl;
    }
};

class Quick : public SortBehavior
{
public:
    void sort() const override
    {
        cout << "Quick Sort()" << endl;
    }
};

class Heap : public SortBehavior
{
public:
    void sort() const override
    {
        cout << "Heap Sort()" << endl;
    }
};
```

main.cpp

Example 2 (SortSearch [Code])

```
// Interface or Abstract class
class SearchBehavior
{
public:
    virtual void search() const = 0;
};
```

main.cpp

```
class Sequential : public SearchBehavior
{
public:
    void search() const override
    {
        cout << "Sequential Search()" << endl;
    }
};

class BinaryTree : public SearchBehavior
{
public:
    void search() const override
    {
        cout << "BinaryTree Search()" << endl;
    }
};

class HashTable : public SearchBehavior
{
public:
    void search() const override
    {
        cout << "HashTable Search()" << endl;
    }
};
```

main.cpp

Example 2 (SortSearch [Code])

// Context

```
class Collection
```

```
{
```

```
public:
```

```
    Collection() {}
```

```
    void set_sort(SortBehavior *s)
```

```
    {
```

```
        m_sort = s;
```

```
    }
```

```
    void set_search(SearchBehavior *s)
```

```
    {
```

```
        m_search = s;
```

```
    }
```

```
    void sort() const
```

```
    {
```

```
        m_sort->sort();
```

```
    }
```

```
    void search() const
```

```
    {
```

```
        m_search->search();
```

```
    }
```

```
private:
```

```
    SortBehavior *m_sort;
```

```
    SearchBehavior *m_search;
```

```
};
```

main.cpp

```
int main()
```

```
{
```

```
    Merge merge;
```

```
    Quick quick;
```

```
    Heap heap;
```

```
    Sequential sequential;
```

```
    BinaryTree binaryTree;
```

```
    HashTable hashTable;
```

```
    Collection colA;
```

```
    colA.set_sort(&merge);
```

```
    colA.sort();
```

```
    Collection colB;
```

```
    colB.set_search(&binaryTree);
```

```
    colB.search();
```

```
}
```

main.cpp

Output:

Merge Sort()

BinaryTree Search()

Example 3 (RecordKeeper[Code])

record.h

```
#pragma once
#include <string>
using namespace std;

// Interface or Abstract Class
class Record
{
public:
    virtual void start_record() = 0;
    virtual void store_field(const string &name, const string &value)
    = 0;
    virtual void finish_record() = 0;
    virtual ~Record() {}
};
```

contactRecord.h

```
#pragma once
#include "record.h"
#include <cassert>

struct ContactData
{
    string first_name, last_name, phone, email;
};
```

Example 3 (RecordKeeper[Code])

```
class ContactRecorder
{
public:
    ContactRecorder(Record *a) : m_record(a)
    {
        assert(a != nullptr); contactRecord.h
    }

    void store(const ContactData &data)
    {
        assert(m_record != nullptr);

        m_record->start_record();
        m_record->store_field("first name", data.first_name);
        m_record->store_field("last name", data.last_name);
        m_record->store_field("phone", data.phone);
        m_record->store_field("email", data.email);
        m_record->finish_record();
    }

private:
    Record *m_record;
};
```

```
#pragma once
#include "record.h"
#include <iostream>
using namespace std; databaseRecord.h

class MySql
{
}; // Dummy class for MySQL connection simulation

class DatabaseRecord : public Record
{
public:
    DatabaseRecord() : m_dbConnection(new MySql) {}

    void start_record() override
    {
        cout << "Start transaction" << endl;
    }

    void store_field(const string &name, const string &value)
    {
        cout << "Insert into table: " << name << " = " << value << endl;
    }

    void finish_record()
    {
        cout << "Finish transaction" << endl;
    }

private:
    MySql *m_dbConnection;
};
```

Example 3 (RecordKeeper[Code])

```
#pragma once
#include "record.h"
#include <iostream>
```

streamRecord.h

```
class StreamRecord : public Record
{
public:
    StreamRecord(ostream &s, const string &record_name = string()) : m_ostream(s),
        m_record_name(record_name) {}

    void start_record() override
    {
        m_ostream << m_record_name << "( ";
    }

    void store_field(const string &name, const string &value) override
    {
        m_ostream << name << ": " << value << "; ";
    }

    void finish_record() override
    {
        m_ostream << ")" << endl;
    }

    void set_record_name(const string &name)
    {
        m_record_name = name;
    }

private:
    ostream &m_ostream;
    string m_record_name;
};
```

```
#include "record.h"
#include "databaseRecord.h"
#include "contactRecorder.h"
#include "streamRecord.h"
```

main.cpp

```
int main()
{
    ContactData data = {"William", "Smith", "415-675-1874", "smith@email.com"};
    cout << "\nStreamRecorder: " << endl;

    StreamRecord sRecord(std::cout, "Contact");
    ContactRecorder contact(&sRecord);
    contact.store(data);

    cout << "\nDatabase Recorder: " << endl;

    DatabaseRecord dbRecord;
    ContactRecorder contact2(&dbRecord);
    contact2.store(data);
}

StreamRecorder:
Contact( first name: William; last name: Smith; phone: 415-675-1874; email: smith@email.com; )

Database Recorder:
Start transaction
Insert into table: first name = William
Insert into table: last name = Smith
Insert into table: phone = 415-675-1874
Insert into table: email = smith@email.com
Finish transaction
```

Example 3 (RecordKeeper[Code])

Output:

StreamRecorder:

Contact(first name: William; last name: Smith; phone: 415-675-1874; email: smith@email.com;)

Database Recorder:

Start transaction

Insert into table: first name = William

Insert into table: last name = Smith

Insert into table: phone = 415-675-1874

Insert into table: email = smith@email.com

Finish transaction